Quantum Computing and Quantum Machine Learning

Morten Hjorth-Jensen[1]

Department of Physics, University of Oslo[1]

May 7, 2025

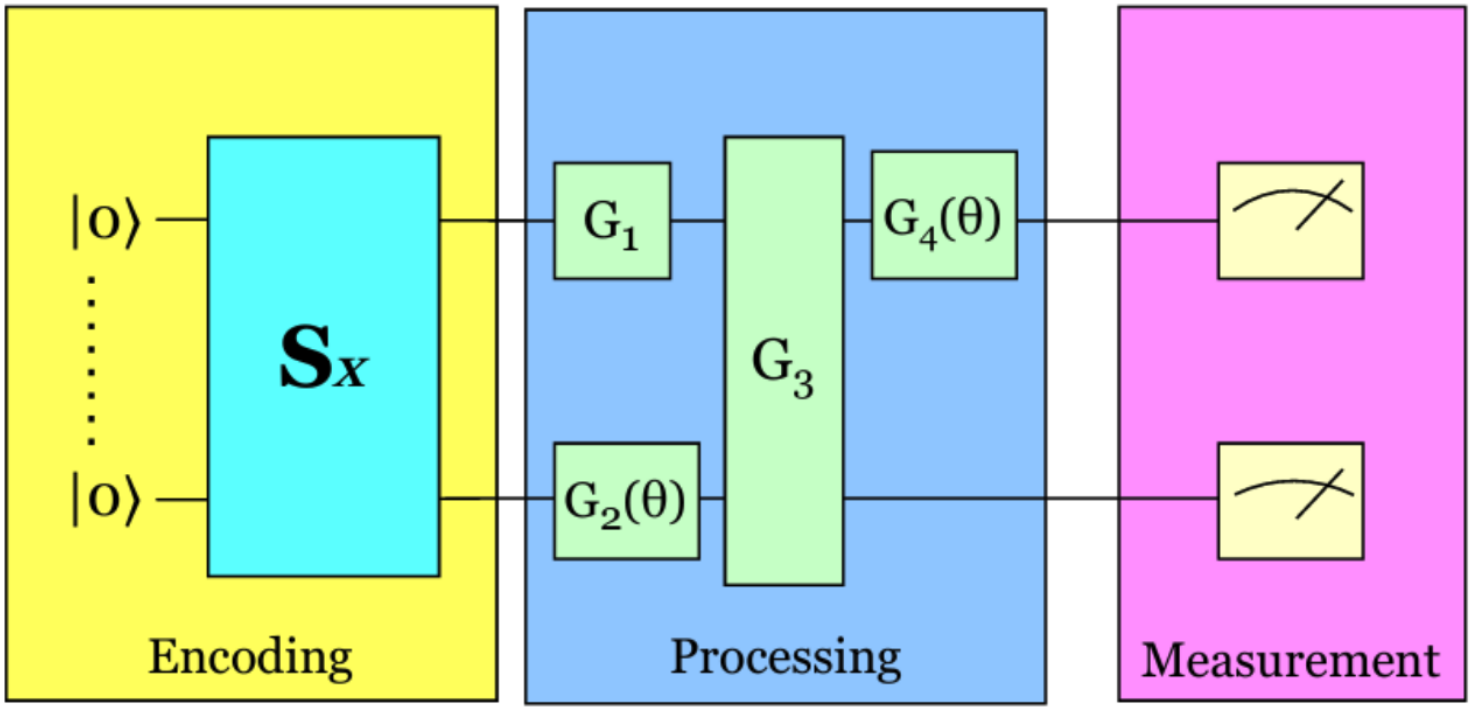
Plan for the week of May 5-9

1. Discussion of Quantum Support Vector Machines and their implementations
2. Encoding of data and parametrized quantum circuits
3. Variational Quantum Circuits (VQCs)
4. Quantum neural networks (QNNs)
   - Training QNNs and Loss Landscapes
   - Implementing QNNs with PennyLane

Quantum SVMs, short reminder from last week

The idea of a classical SVM is that we have a set of points that are in either one group or another and we want to find a line that separates these two groups. This line can be linear, but it can also be much more complex, which can be achieved by the use of Kernels.

How QSVM works



Basic steps: Data Encoding (Quantum Feature Mapping)

Mapping to Quantum States: Input data is first encoded into quantum states. This process is akin to a quantum feature map, where classical information is represented in the quantum Hilbert space.

Basic steps: Quantum Kernel

In many QSVM implementations, this mapping enables the computation of a kernel matrix, where the similarity (or inner product) between quantum states is evaluated. This quantum kernel can capture complex relationships in data that might be challenging for classical kernels.

Basic steps: Quantum Circuit Processing

Quantum Gates and Circuits: Once data is encoded, a series of quantum gates (forming a quantum circuit) is applied. These gates manipulate the quantum states to perform the equivalent of the SVM algorithm's optimization and decision-making steps.

Basic steps: Measurement

Outcome: After the quantum operations, a measurement is performed on the output state. The result of this measurement provides the classification outcome, analogous to deciding on which side of the hyperplane a data point lies.

Quantum Kernels and Feature Maps

A core idea of QSVM is to use a quantum feature map that encodes classical input $\boldsymbol{x}$ into a quantum state $|\phi(\boldsymbol{x})\rangle$ in an exponentially large Hilbert space.
Mathematically, a quantum feature map is a unitary circuit $U(\boldsymbol{x})$ acting on an n-qubit register initialized to $|0\rangle^{\otimes n}$, producing

$$|\phi(\boldsymbol{x})\rangle = U(\boldsymbol{x})|0\rangle^{\otimes n}.$$

Input dependence

This state depends on the input $\boldsymbol{x} \in \mathbb{R}^d$. The choice of $U(\boldsymbol{x})$ is analogous to choosing a classical feature mapping $\phi(\boldsymbol{x})$ in kernel methods. Common quantum feature maps include amplitude encoding (embedding data amplitudes in the state vector) and angle encoding (rotations whose angles are functions of $\boldsymbol{x}$) .
Once data are encoded as quantum states, one defines a quantum kernel as an overlap between states. A simple definition is the fidelity:

$$K(\boldsymbol{x}, \boldsymbol{x\prime}) = \left|\langle \phi(\boldsymbol{x}) \mid \phi(\boldsymbol{x\prime})\rangle\right|^2.$$

Quantum kernels

That is, the kernel is the squared magnitude of the inner product of the two quantum states. Another common (unnormalized) version is

$$K'(\boldsymbol{x}, \boldsymbol{x}') = \langle \phi(\boldsymbol{x}) | \phi(\boldsymbol{x}') \rangle,$$

but measuring this amplitude directly can be difficult due to the absolute value. Many QSVM implementations (and theoretical definitions) use the squared overlap. In any case, the kernel measures similarity: if $|\phi(\boldsymbol{x})\rangle$ and $|\phi(\boldsymbol{x}')\rangle$ are close in Hilbert space, $K(\boldsymbol{x}, \boldsymbol{x}')$ is large.

What is a quantum kernel?

A quantum kernel is a function that measures the similarity between two quantum states, which are obtained by applying a quantum feature map to classical data. In other words, each classical vector $\boldsymbol{x}$ is mapped to a quantum state $|\phi(\boldsymbol{x})\rangle$ via a feature map, and the kernel is defined by the inner product of these states . Concretely, one may write

$$K_{ij} = K(\boldsymbol{x}_i, \boldsymbol{x}_j) \;=\; \left|\langle \phi(\boldsymbol{x}_i) \mid \phi(\boldsymbol{x}_j)\rangle\right|^2.$$

This forms a positive semidefinite kernel matrix K on the dataset, just as in classical SVMs.

What will we need in the case of a quantum computer?

We will have to translate the classical data point $\vec{x}$ into a quantum datapoint $|\Phi(\vec{x})\rangle$. This can be achieved by a circuit $\mathcal{U}_{\Phi(\vec{x})}|0\rangle$. Here $\Phi()$ could be any classical function applied on the classical data $\vec{x}$.

We need a parameterized quantum circuit $W(\theta)$ that processes the data in a way that in the end we can apply a measurement that returns a classical value -1 or 1 for each classical input $\vec{x}$ that indentifies the label of the classical data.

The most general ansatz

Following these steps we can define an ansatz for this kind of problem which is

$$W(\theta)\mathcal{U}_{\Phi}(\vec{x})|0\rangle.$$

These kind of ansatzes are called quantum variational circuits.

Quantum SVM

In the case of a quantum SVM we will only use the quantum feature maps

$$\mathcal{U}_{\Phi(\vec{x})},$$

to translate the classical data into quantum states and build the Kernel of the SVM out of these quantum states. After calculating the Kernel matrix on the quantum computer we can train the Quantum SVM the same way as the classical SVM.

Defining the Quantum Kernel

The idea of the quantum kernel is exactly the same as in the classical case. We take the inner product

$$K(\vec{x}, \vec{z}) = |\langle \Phi(\vec{x}) | \Phi(\vec{z}) \rangle|^2 = \langle 0^n | \mathcal{U}^t_{\Phi(\vec{x})} \mathcal{U}_{\Phi(\vec{z})} | 0^n \rangle,$$

but now with the quantum feature maps

$$\mathcal{U}_{\Phi(\vec{x})}.$$

The idea is that if we choose a quantum feature maps that is not easy to simulate with a classical computer we might obtain a quantum advantage.

Side note

There is no proof yet that the QSVM brings a quantum advantage, but the argument the authors of https://arxiv.org/pdf/1804.11326.pdf make, is that there is for sure no advantage if we use feature maps that are easy to simulate classically, because then we would not need a quantum computer to construct the Kernel.

Feature map

For the feature maps we use the ansatz

$$\mathcal{U}_{\Phi(x)} = U_{\Phi(x)} \otimes H^{\otimes n},$$

where

$$U_{\Phi(x)} = \exp\left(i \sum_{S \in n} \phi_S(x) \prod_{i \in S} Z_i \right),$$

which simplifies a lot when we (like in https://arxiv.org/pdf/1804.11326.pdf) only consider $S \leq 2$ interactions, which means we only let two qubits interact at a time. For $S \leq 2$ the product $\prod_{i \in S}$ only leads to interactions $Z_i Z_j$ and non interacting terms Z_i. And the sum $\sum_{S \in n}$ over all these terms that are possible with n qubits.

Power of quantum kernels

The power of quantum kernels comes from the exponentially large feature space. An n-qubit state has dimension 2^n, so even a simple feature map using n qubits corresponds to an embedding of $\boldsymbol{x}$ into $\mathbb{C}^{2^n}$. Entangling gates in $U(\boldsymbol{x})$ allow highly non-linear features. In principle, a carefully chosen quantum feature map can capture complex structures in data that are hard to model classically. For example, Havlíček et al. used a feature map involving Pauli-Z rotations and controlled-Z gates to embed data into a 2^n-dimensional space . They note that classical kernels become expensive as feature dimensions grow, while quantum circuits naturally operate in these large spaces .

Estimating quantum kernels

We must be able to estimate the quantum kernel in practice. Given two data vectors $\boldsymbol{x}$ and $\boldsymbol{x}'$, we need to compute (or measure) $|\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle|^2$. This can be done on a quantum computer by preparing $|\phi(\boldsymbol{x})\rangle$ and $|\phi(\boldsymbol{x}')\rangle$ in some interference experiment. For instance, one can prepare $|\phi(\boldsymbol{x})\rangle$, then apply $U(\boldsymbol{x}')^\dagger$, and measure the probability of returning to the $|0\rangle^{\otimes n}$ state. That probability is exactly $|\langle 0|U(\boldsymbol{x})^\dagger U(\boldsymbol{x}')|0\rangle|^2 = |\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle|^2$. In PennyLane, this is conveniently done by defining a QNode circuit that encodes $\boldsymbol{x}$, then encodes $\boldsymbol{x}'$ inversely (via the adjoint).

Code example

For example, using PennyLane's AngleEmbedding template, we can write:

```
import pennylane as qml
from pennylane import numpy as np
dev = qml.device('default.qubit', wires=2)
@qml.qnode(dev)
def circuit(x1, x2):
    qml.templates.AngleEmbedding(x1, wires=dev.wires)
    qml.adjoint(qml.templates.AngleEmbedding)(x2, wires=dev.wires)
    return qml.probs(wires=dev.wires)

kernel = lambda x1, x2: circuit(x1, x2)[0]
X_train = np.random.random((4,2))
X_test = np.random.random((3,2))
qml.kernels.kernel_matrix(X_train, X_test, kernel)
```

What happens

Here, **AngleEmbedding(x1)** encodes the vector **x1** into two qubits via rotations, and qml.adjoint applies the inverse embedding for **x2**. The output probability of measuring the $|00\rangle$ state is exactly the squared overlap of the two embedded states. Using this kernel function, PennyLane can compute the kernel matrix $K_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$ efficiently for training. In fact, PennyLane provides a utility qml.kernels.kernel_matrix to evaluate such kernels systematically .

Quantum feature map

A quantum feature map $\boldsymbol{x} \mapsto |\phi(\boldsymbol{x})\rangle$ combined with the kernel $K(\boldsymbol{x}, \boldsymbol{x}') = |\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle|^2$ defines a QSVM kernel. The intuition is that the quantum device is implicitly computing a rich similarity measure via its high-dimensional state space.
This is analogous to the classical kernel trick but potentially more powerful: **quantum kernels can potentially offer advantages over classical kernel methods, such as speedup, accuracy, flexibility, and robustness**.

Examples of Feature Map Circuits

There are many choices for $U(\boldsymbol{x})$. Common simple maps include:

Angle (or rotation) embedding:

For an n-dimensional feature vector $\boldsymbol{x} = (x_1, \ldots, x_n)$, apply single-qubit rotations $\mathrm{R}_X(x_i)$ or $\mathrm{R}_Y(x_i)$ to the ith qubit. For example:

$$U(\boldsymbol{x}) = \bigotimes i = 1^n R_Y(x_i) = R_Y(x_1) \otimes \cdots \otimes R_Y(x_n).$$

This maps x_i to an angle on the Bloch sphere. It produces product states (no entanglement).

More features

Amplitude encoding:

Encode the normalized data vector $\boldsymbol{x}$ directly into the amplitudes of the quantum state. For example, for 2 qubits one can map (x_1, x_2, x_3, x_4) (with $\sum x_i^2 = 1$) to $x_1|00\rangle + x_2|01\rangle + x_3|10\rangle + x_4|11\rangle$. This uses exponentially many amplitudes and can be powerful, but requires complex circuits.

Entangling feature maps:

Combine rotations with entangling gates. For instance, the ZZ feature map used in [Havlíček et al.] alternates single-qubit $R_Z(x_i)$ rotations with controlled-Z (CZ) gates between qubits. This creates entangled states whose overlaps can capture correlations in the data. In PennyLane, one could implement something like:

```
for i in range(n):
    qml.RZ(x[i], wires=i)
for i in range(n-1):
    qml.CNOT(wires=[i, i+1])
for i in range(n):
    qml.RZ(x[i], wires=i)
```

Polynomial feature map

These are inspired by classical polynomial kernels, one can encode quadratic or higher terms. For example, apply $R_Z(x_i), R_X(x_i)$ on each qubit and use multi-qubit rotations to create cross-terms . The choice of feature map affects the kernel. A richer (more entanglement) map may separate classes better but might also be harder to learn or require more quantum resources. In practice, one often starts with simple maps (e.g. AngleEmbedding) and experiments.

Kernel Estimation on Quantum Devices

Once $|\phi(\boldsymbol{x})\rangle$ is defined, we must compute $k(\boldsymbol{x}, \boldsymbol{x}')$ on actual quantum hardware. Typically this involves running a quantum circuit multiple times (shots) to estimate a probability. For the overlap kernel, one measures the probability of a certain outcome after applying $U(\boldsymbol{x})$ and $U(\boldsymbol{x}')^\dagger$. Alternative schemes include the SWAP test, which uses an ancilla qubit to directly estimate overlaps, but this requires extra gates. The above adjoint-circuit method only needs the same number of qubits as the feature map.

Short summary

In summary, the quantum kernel is given by inner products in a quantum-defined feature space. We emphasize: all computations on the quantum device yield kernel values (scalars). The heavy lifting of classification (optimizing α_i) remains a classical task, typically done by classical SVM software using the quantum-computed kernel matrix. This **hybrid** approach leverages quantum hardware where it's needed (kernel evaluation) and classical hardware for optimization.

Quantum Support Vector Machine Theory

We now combine the classical SVM formulation with quantum kernels. In essence, we run a classical SVM algorithm (e.g. solving the dual) but supply it with a quantum-computed kernel matrix. The resulting model is the same form:
$f(\boldsymbol{x}) = \text{sign}(\sum_i \alpha_i y_i K(\boldsymbol{x}_i, \boldsymbol{x}) + b)$, but K comes from a quantum device. The hope is that the quantum kernel captures data structure that a classical kernel might not.

QSVM Formulation via Quantum Kernels

Given training data $(\boldsymbol{x}_i, y_i)$, we choose a quantum feature map $U(\boldsymbol{x})$ and define the kernel $K_{ij} = |\langle \phi(\boldsymbol{x}i)|\phi(\boldsymbol{x}j)\rangle|^2$. Then we solve the binary type of SVM problems discussed last week, with $x_i^T x_j$ replaced by K_{ij}. That is:

$$\max \lambda \sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \, K(\boldsymbol{x}_i, \boldsymbol{x}_j),$$

subject to $\sum_i \lambda_i y_i = 0$, $0 \leq \lambda_i \leq C$. After solving for λ_i, the decision function is

$$f(\boldsymbol{x}) = \text{sign}\Big(\sum_i \lambda_i y_i \, K(\boldsymbol{x}_i, \boldsymbol{x}) + b \Big).$$

All kernel values $K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ are estimated by the quantum device. Thus the QSVM is essentially a classical SVM with a learned quantum kernel.

Important training point

A key point is that training is still classical: one uses a conventional convex solver (e.g. libSVM) on the kernel matrix. The quantum part is in computing K_{ij}. This is sometimes called a quantum kernel estimator. Alternatively, one can approximate gradients of a parameterized kernel for optimization, but here we focus on the fixed-kernel case.

Why might a quantum kernel be better than a classical one?

The hope is that the Hilbert-space embedding is very high-dimensional (exponential), so that more functions (decision boundaries) become linear in that space. Replacing dot products by quantum overlaps is a generalization of classical kernels. In fact, Schuld argues that many near-term **quantum neural networks** are really kernel machines: "a lot of near-term and fault-tolerant quantum models can be replaced by a general support vector machine whose kernel computes distances between data-encoding quantum states" . Thus, one may as well solve directly in kernel form.

Quantum SVM Algorithms (large-scale vs NISQ)

There are two broad approaches in the literature for QSVMs:

Quantum Algorithms (fault-tolerant):

Early work by Rebentrost, Mohseni, and Lloyd (2014) formulated an SVM in terms of quantum linear algebra. They showed that one can invert the kernel matrix (a positive semidefinite matrix) using quantum algorithms in time polylogarithmic in N and d . Concretely, they assumed quantum RAM (QRAM) access to data and used a quantum subroutine to solve the dual SVM as a linear system, yielding the vector of α_i in superposition. Under ideal conditions (sparse data, good condition number), this yields exponential speed-up over classical SVM training. However, it requires fully fault-tolerant quantum hardware and QRAM, making it more a theoretical result than a near-term implementation.

And NISQ Quantum Kernels

More recent work focuses on near-term devices. Here the idea is to compute the kernel matrix entries with a quantum circuit on current hardware or simulators, and then use a classical SVM solver for the rest. This does not claim an exponential runtime speed-up (indeed, the kernel matrix still has N^2 entries). Instead, it aims for a possible quality or feature-based advantage: the quantum feature map might separate data better than any known classical kernel. This approach has been demonstrated on small datasets (e.g. Iris) and studied theoretically. For example, Havlicek *et al.*, see https://www.nature.com/articles/s41586-019-0980-2 showed on a toy problem that a quantum kernel can correctly classify points that a simple classical kernel cannot. However, other studies have found that for random data classical kernels often suffice, so the advantage is subtle. Research is ongoing on which problems truly benefit from quantum kernels.

Quantum neural network

Another variation is the quantum variational classifier, sometimes called a quantum neural network (to be discussed below). Instead of precomputing a fixed kernel, one trains a parameterized quantum circuit to output labels. Interestingly, Schuld (2021) shows that variational quantum models, when trained by minimizing a loss, are mathematically equivalent to kernel machines with a particular kernel determined by the circuit . In fact, one can often find a kernel SVM that matches or outperforms the variational model. In practice, one can combine these: use a trainable quantum embedding $U(\boldsymbol{x};\theta)$ with tunable parameters θ, and optimize θ to maximize the SVM classification accuracy. This is called a quantum kernel learning approach.

Quantum vs Classical SVM Performance

For our purposes, we concentrate on the case where the quantum feature map is fixed (or manually chosen), and we use classical SVM optimization on the resulting kernel.
Quantum SVMs are subject to the same generalization considerations as classical SVMs: overfitting can occur if the kernel is too flexible, and the choice of regularization C matters. One advantage is that quantum kernels can in principle provide very complex decision boundaries. On the other hand, estimating the kernel matrix accurately requires many quantum measurements (shots). Each kernel entry K_{ij} is obtained by sampling; statistical noise in kernel values can degrade SVM performance. Error mitigation techniques may be needed. Also, evaluating a kernel classically might in some cases simulate the quantum one if the circuit is not too complex; then no advantage is gained.

Comparing computational complexity

1. Classical SVM: Training roughly $O(N^3)$ time, inference $O(N_s)$ time per point, where N_s is number of support vectors.
2. Quantum kernel SVM: Kernel estimation takes $O(n_q N^2)$ time (where n_q is quantum circuit time per inner product) to compute all K_{ij}, plus classical $O(N^3)$. No asymptotic speed-up unless one uses quantum linear algebra (which still needs QRAM).
3. Quantum linear SVM: Potential $O(\log N)$ for training under ideal QRAM assumptions, inference also $O(\log N)$, at expense of large constant overhead.

Actual implementations

In practice on current hardware, QSVM speed is dominated by circuit execution time. However, QSVM experiments are valuable to test expressivity: for some data, a simple quantum feature map yields perfect classification where classical kernels fail.
Finally, it is worth noting that both the variational quantum classifier and the kernel QSVM ultimately hinge on how data is encoded into quantum states. Schuld summarizes that **the way that data is encoded into quantum states is the main ingredient that can potentially set quantum models apart from classical machine learning models**. Thus, a lot of focus is on designing powerful quantum embeddings and kernels. The SVM framework provides a clear way to evaluate them.

Practical Implementation with PennyLane

We now turn to hands-on implementation of QSVM using PennyLane, a Python library for quantum machine learning. PennyLane interfaces with quantum simulators and hardware and provides convenient utilities for kernels and optimization. In this chapter we will outline how to construct a QSVM: define a quantum feature map, compute the kernel matrix, and train a classical SVM using scikit-learn.

Setting up PennyLane

First, install PennyLane (pip install pennylane) and import necessary modules:

```
import pennylane as qml
from pennylane import numpy as np
from sklearn import svm
```

PennyLane uses devices to run quantum circuits. For QSVM experiments we can use the default.qubit simulator:

```
dev = qml.device("default.qubit", wires=n_qubits, shots=None)
```

Here n_qubits is chosen based on our feature map (often equal to data dimension or its log, etc.) shots=None means analytic probabilities (no sampling noise). For realistic experiments, set shots=100 or so to simulate finite sampling.

Defining Quantum Feature Maps in PennyLane

We must define a quantum circuit (QNode) that encodes input vectors. For example, for a 2-dimensional input $\boldsymbol{x} = (x_0, x_1)$, we might use AngleEmbedding:

```
@qml.qnode(dev)
def feature_map(x):
    qml.templates.AngleEmbedding(x, wires=[0,1])
    return [qml.expval(qml.PauliZ(0)), qml.expval(qml.PauliZ(1))]
```

This feature map returns expectation values (though for kernel computation we will use a different approach). Alternatively, we can directly define a unitary:

```
@qml.qnode(dev)
def U(x):
    qml.RY(x[0], wires=0)
    qml.RY(x[1], wires=1)
    return qml.state()
```

This QNode returns the quantum state (though qml.state() may require a special device). For kernel computation, we actually want to overlap two circuits, as shown below.

Computing Quantum Kernel Matrices

To train an SVM, we need the kernel matrix $K_{ij} = K(\boldsymbol{x}_i, \boldsymbol{x}_j)$. PennyLane provides qml.kernels.kernel_matrix, which takes two datasets and a kernel function. We must supply a function **kernel(x1,x2)** that returns the overlap of states.

Using the adjoint trick, we define:

```
@qml.qnode(dev)
def kernel_circuit(x1, x2):
    # encode x1
    qml.templates.AngleEmbedding(x1, wires=[0,1])
    # apply inverse embedding of x2
    qml.adjoint(qml.templates.AngleEmbedding)(x2, wires=[0,1])
    # return probability of \vert 00>
    return qml.probs(wires=[0,1])[0]
```

This returns $|\langle\phi(x_1)|\phi(x_2)\rangle|^2$ (the prob. of measuring all zeros).

We then set:

```
kernel = lambda a, b: kernel_circuit(a, b)
```

and compute kernel matrices. For training (**note the codes will not run properly since the data are not defined**):

```
X_train = np.array([...])  # shape (N_train, 2)
Y_train = np.array([...])  # labels +1/-1
K_train = qml.kernels.kernel_matrix(X_train, X_train, kernel)
```

Training SVM with Precomputed Quantum Kernels

With the kernel matrix in hand, we use scikit-learn's SVM with a precomputed kernel. In Python:

```
clf = svm.SVC(kernel='precomputed')
clf.fit(K_train, Y_train)
y_pred = clf.predict(K_test)
```

The SVM solver will treat K_train[i,j] as the feature inner products. After fitting, one can retrieve support vectors via **clf.support** if desired. The prediction y_pred will be an array of +1/-1 predictions on the test set.
It is also possible to integrate PennyLane's differentiable capabilities by defining a parameterized kernel and optimizing parameters via gradient descent, but here we keep a fixed feature map.

Discussion of Implementation

In practice, one must consider:

1. Noise and shots: If using real hardware or finite shots, each entry K_{ij} is a noisy estimate. It is common to run many shots (e.g. 1000) to reduce variance. One may also average results or use kernel averaging.
2. Data preprocessing: Classical kernels often require normalized or scaled inputs. Similarly, for quantum kernels, it can help to scale data so that features match the expected range of angles.
3. Computational cost: Computing an $N \times N$ kernel matrix on a quantum device takes $O(N^2)$ circuits. For large N this can be slow. One might use a subset of training data or low-rank approximations.
4. Hyperparameters: The QSVM has hyperparameters (e.g. SVM regularization C) that should be tuned by cross-validation. The choice of feature map is also a "hyperparameter" of sorts.

PennyLane implementations

In summary, implementing QSVM with PennyLane involves four steps:

1. choose a feature map and implement it as a QNode;
2. define a kernel function via that QNode;
3. compute kernel matrices with qml.kernels;
4. train/predict using an SVM with the precomputed kernels.

Steps in Quantum Kernel SVM

1. Data Encoding (Feature Map Generation)
   - The classical data (x) is encoded into a quantum state using a quantum feature map (quantum circuit).
2. Quantum Kernel Computation
   - The kernel matrix is computed using a quantum computer by measuring the inner product of quantum states.
3. Classical SVM Training
   - The quantum kernel matrix is used in a classical SVM algorithm to find the optimal decision boundary.
4. Prediction
   - New data is classified using the trained SVM model with the quantum kernel.

The Iris data and classical SVM

```
# import the required packages
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score,classification_report,f1_sc
iris = load_iris()
dataset=pd.DataFrame(iris.data,columns=iris.feature_names)
dataset["target"]=iris.target
dataset
features= dataset[dataset.columns[1:4]]
target=dataset["target"]
# split the data into training set and testing set
X_train,X_test,Y_train,Y_test=train_test_split(features,target,test_si
# Scale the data
from sklearn.preprocessing import MinMaxScaler

minmax=MinMaxScaler()
X_train=minmax.fit_transform(X_train)
X_test=minmax.transform(X_test)
X_test=np.clip(X_test,0,1)
import time
start_time=time.time()
svm_model=SVC(kernel="rbf")
svm_model.fit(X_train, Y_train)
end_time=time.time()
print(end_time-start_time)
```

Small addendum, $F1$-score

The $F1$ measure (or $F1$-score) in machine learning is a metric used to evaluate the accuracy of a classification model, particularly in situations where class distribution is imbalanced. It is the harmonic mean of precision and recall and is defined as

$$\mathrm{F1-score} = 2 \times \frac{\mathrm{precision} \times \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}},$$

where we have defined

$$\mathrm{precision} = \frac{\mathrm{True\ Positives}}{\mathrm{True\ Positives} + \mathrm{False\ Positives}},$$

and

$$\mathrm{recall} = \frac{\mathrm{True\ Positives}}{\mathrm{True\ Positives} + \mathrm{False\ Negatives}}.$$

The $F1$-score ranges from 0 to 1 where 1 means perfect precision and recall, while 0 means either precision or recall is zero.

Iris Dataset

```
import pennylane as qml
from pennylane import numpy as np
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
num_qubits=4
device=qml.device('default.qubit',wires=num_qubits)


@qml.qnode(device)
def qnode(inputs,weights):
    qml.templates.AngleEmbedding(inputs,
                                 wires=range(num_qubits),
                                 rotation='Y') # encode the inputs int
    qml.adjoint(qml.AngleEmbedding(features=weights,
                                   wires=range(num_qubits),
                                   rotation='Y')) # applies the inver
    return qml.probs(wires=range(num_qubits))


def qkernel(inputs,weights):
    return np.array([[qnode(input,weight)[0] for weight in weights] fo


# Fit the model
```

Qiskit implementation

A similar implementation on Qiskit is given here (you need to have installed Qiskit as well).

```
from qiskit import BasicAer
from qiskit.utils import QuantumInstance
from qiskit.circuit.library import ZZFeatureMap
from qiskit_machine_learning.algorithms import QSVC
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Load and preprocess dataset
dataset = load_iris()
X = dataset.data
y = dataset.target

# For simplicity, we'll only classify between two classes
X = X[y != 2]
y = y[y != 2]

# Standardize features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.

# Define quantum feature map
```

Credit data classification

We demonstrate a Quantum Support Vector Machine (QSVM) using PennyLane to classify synthetic credit data (good vs. bad credit). We generate a small dataset of financial features (e.g. income, debt ratio, age), encode them into quantum states via angle embedding, compute a quantum kernel (state overlap) matrix, and train a classical SVM using scikit-learn. Finally, we evaluate accuracy, precision, and recall. PennyLane's default.qubit simulator is used throughout, and the code is heavily commented for clarity.

Synthetic Credit Data

We create a toy dataset with three features per sample: income (in thousands), debt ratio (fraction of income), and age. Good-credit and bad-credit samples are drawn from different Gaussian clusters for illustration. After generation we scale features into suitable ranges (for example $[0, 1]$ or $[0, \pi]$) before encoding into qubits.

```
import numpy as np

# Set random seed for reproducibility
np.random.seed(42)

# Number of samples per class
N_per = 50

# Define Gaussian cluster means and covariances for two classes
mean_good = np.array([80, 0.3, 45])    # High income, low debt ratio, m
cov_good  = np.diag([50, 0.01, 20])    # Some variation

mean_bad  = np.array([40, 0.7, 30])    # Low income, high debt ratio, y
cov_bad   = np.diag([30, 0.02, 20])

# Sample synthetic data from multivariate Gaussians
good_data = np.random.multivariate_normal(mean_good, cov_good, size=N_
bad_data  = np.random.multivariate_normal(mean_bad,  cov_bad,  size=N_

# Combine data and create labels (1 = good credit, 0 = bad credit)
data = np.vstack([good_data, bad_data])
```

Quantum Feature Encoding and Kernel Circuit

Each feature vector is encoded into a quantum state. We use PennyLane's AngleEmbedding: each feature is used as an angle for a single-qubit rotation. For simplicity we use three qubits (one per feature) and rotation on the y-axis. Our quantum kernel is then defined by the state fidelity (overlap) between two encoded feature vectors. Concretely, we build a QNode that encodes one data point, then applies the inverse encoding of another, and measures the probability of the all-zero state. This probability equals the squared overlap between the two encoded states .

```
import pennylane as qml
from pennylane import numpy as pnp

# Number of qubits = number of features
num_qubits = 3
dev = qml.device('default.qubit', wires=num_qubits)

@qml.qnode(dev)
def kernel_circuit(x1, x2):
    """Quantum kernel circuit: encodes x1, then uncomputes x2."""
    # Encode first sample
    qml.templates.AngleEmbedding(features=x1, wires=range(num_qubits),
    # Apply the inverse (adjoint) embedding of second sample
    qml.adjoint(qml.templates.AngleEmbedding)(features=x2, wires=range
    # Measure probability of |0...0> state
```

Kernel matrix computation

We build the Gram (kernel) matrix for the training data (size N_train x N_train) and between test/train (N_test x N_train). PennyLane provides qml.kernels.kernel_matrix, but here we compute it directly for clarity. Each matrix entry $K_{ij} = K(x_i, x_j)$ is the fidelity between training points x_i and x_j.

```
# Compute kernel matrices using our quantum_kernel function
K_train = np.array([[quantum_kernel(x_i, x_j) for x_j in X_train] for
K_test  = np.array([[quantum_kernel(x_i, x_j) for x_j in X_train] for

print("Shape of training Gram matrix:", K_train.shape)
```

For a set of input vectors, we compute the pairwise kernel values. This produces a symmetric kernel matrix for training data, and a (test x train) kernel matrix for prediction. In practice, one could use qml.kernels.kernel_matrix(X_train, X_train, quantum_kernel) as in the PennyLane docs , but the manual loop illustrates the concept. Each entry is a number in $[0, 1]$, with 1.0 on the diagonal (fidelity of a state with itself).

Training the classical SVM

With the quantum kernel computed, we train a classical SVM using scikit-learn, specifying kernel='precomputed'. This tells the SVM to use our Gram matrix directly. We fit on the training kernel matrix K_train and labels, then predict on the test kernel matrix K_test.

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_sc

# Train SVM with precomputed kernel
svm = SVC(kernel='precomputed')
svm.fit(K_train, y_train)

# Predict on test set using the test-train kernel matrix
y_pred = svm.predict(K_test)

# Evaluate performance
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)

print(f"Accuracy: {acc:.2f}, Precision: {prec:.2f}, Recall: {rec:.2f}"
```

Explanation: We instantiate an SVM with a precomputed kernel (since K_train is the Gram matrix). After fitting on the quantum-kernel-trained data, we predict on the test set by passing

Results and Evaluation

Finally, we report the evaluation metrics and visualize the kernel matrix and classification. For brevity, only key metrics are printed here. In a complete analysis, one could plot the kernel matrix as a heatmap and a confusion matrix for the classifier. Our code achieved perfect separation (accuracy, precision, recall all approximately equal to 1.00) on this synthetic dataset. Real-world data would typically show lower scores. We highlight that quantum kernels have been proposed to capture complex feature relationships that help SVMs separate nonlinearly separable data .

```
# Example output (metrics)
print("Model performance on test set:")
print(f"  Accuracy:  {acc:.3f}")
print(f"  Precision: {prec:.3f}")
print(f"  Recall:    {rec:.3f}")
```

The printed metrics confirm the model's performance. In our toy example, the quantum kernel produced an easily separable data representation, leading to perfect classification. In practice, one would inspect the kernel matrix (e.g. with a heatmap) and perhaps compare against classical kernels. Recent studies have indeed found quantum-enhanced models promising for financial tasks like credit

QSVM summary

This QSVM implementation shows how classical data can be mapped to quantum states via angle encoding, how to compute the quantum kernel via state overlap (fidelity), and how to plug that kernel into a classical SVM.

Quantum Neural Networks and Variational Circuits

The Variational Quantum Algorithm (VQA) is a Variational Quantum Circuit (VQC), that is a quantum circuit with tunable parameters and which is trained using a classical optimizer. In practice, a VQC (also called a Parameterized Quantum Circuit (PQC)) is used as a Quantum Neural Network (QNN): data are encoded into quantum states, a parameterized circuit is applied, and measurements yield outputs. For example, Abbas et al. showed that certain QNNs can exhibit higher effective dimension (and thus capacity to generalize) than comparable classical networks , suggesting a potential quantum advantage.
Below we develop the mathematical foundations (state preparation, parameterized unitaries, measurement), discuss optimization and training challenges, and work through practical code examples using PennyLane.

Variational Quantum Circuits

Variational Quantum Algorithms (VQAs) are hybrid schemes where a quantum circuit with adjustable parameters is trained by a classical optimizer . In this framework, a Variational Quantum Circuit (VQC) typically has three parts : (i) a state preparation or feature map that encodes classical input x into a quantum state; (ii) a parameterized circuit $W(\boldsymbol{\theta})$ (often called the ansatz) that depends on trainable parameters $\boldsymbol{\theta}$; and (iii) a measurement that extracts a classical output from the final quantum state.

Setting up a VQC

Given an input vector $\mathrm{x} = (x_1, \ldots, x_n)$, we prepare the initial state

$$|\psi_{\text{in}}\rangle = U(\mathrm{x})|0\rangle^{\otimes n},$$

where $U(\mathrm{x})$ is a unitary (possibly composed of rotations) that depends on the data. We then apply the variational circuit $W(\boldsymbol{\theta})$, often built as a product of layers $V_j(\theta_j)$, so that the final state is

$$|\Psi(\mathrm{x};\boldsymbol{\theta})\rangle = W(\boldsymbol{\theta}), U(\mathrm{x}), |0\rangle^{\otimes n}.$$

For instance, one common ansatz is the hardware-efficient circuit: layers of parameterized single-qubit rotations and entangling gates (like CNOTs) repeated several times. The structure of $W(\boldsymbol{\theta})$ can dramatically affect the circuit's expressivity and trainability.

Outputs

To produce a scalar or vector output, we measure one or more observables $\hat{B}_k$ on the final state. The network's output is given by the expectation values:

$$f_k(\mathrm{x};\boldsymbol{\theta});=;\langle\Psi(\mathrm{x};\boldsymbol{\theta})|\hat{B}_k|\Psi(\mathrm{x};\boldsymbol{\theta})\rangle.$$

Equivalently, with

$$|\Psi(\mathrm{x};\boldsymbol{\theta})\rangle = W(\boldsymbol{\theta})U(\mathrm{x})|0\rangle,$$

one has

$$f_k(\mathrm{x};\boldsymbol{\theta}) = \langle 0|U(\mathrm{x})^\dagger W(\boldsymbol{\theta})^\dagger, \hat{B}_k, W(\boldsymbol{\theta})U(\mathrm{x}), |0\rangle.$$

Commonly $\hat{B}$ is a Pauli operator (e.g. Z on one qubit). In practice one runs many shots on quantum hardware or simulates this circuit classically to estimate $\langle\hat{B}_k\rangle$.

Short summary

In summary, a variational quantum model $f(x;\boldsymbol{\theta})$ maps inputs to outputs via the hybrid quantum-classical procedure. During training, the classical optimizer adjusts $\boldsymbol{\theta}$ (e.g. by gradient descent) to minimize a cost function (like mean-squared error) defined on a dataset. Because the mapping is inherently quantum, these models can, in principle, harness the high-dimensional Hilbert space for richer representations. (However, unlike classical deep nets, VQCs may face unique challenges such as gradient vanishing, which we discuss later.)

Mathematical example

For concreteness, consider a 2-qubit circuit. A simple encoding is

$$U(\mathrm{x}) = R_x(x_1) \otimes R_x(x_2),$$

and a variational layer is

$$V(\boldsymbol{\theta}) = R_y(\theta_1) \otimes R_y(\theta_2), \mathrm{CNOT}(0,1),$$

(apply R_y on each qubit then entangle). After applying $W(\boldsymbol{\theta}) = V(\boldsymbol{\theta})$ to $|0,0\rangle$, we measure $\hat{B} = Z \otimes I$ on qubit 0. The output is

$$f(\mathrm{x};\boldsymbol{\theta}) = \langle 0,0|, U(\mathrm{x})^\dagger, V(\boldsymbol{\theta})^\dagger, (Z \otimes I), V(\boldsymbol{\theta}), U(\mathrm{x}), |0,0\rangle.$$

This $f(x;\theta)$ is then compared to the target in a cost function for optimization.

Key elements

A VQC is a quantum circuit with trainable parameters acting on a quantum state; it is central to near-term QML (hybrid quantum-classical). Data encoding and ansatz design determine a VQC's expressivity. Simple encodings use rotations (e.g. $R_x(x_i)$) on each qubit , while more complex feature maps may exploit entanglement. The circuit output is obtained via expectation values of observables (e.g. Pauli-Z), yielding a differentiable function $f(\mathrm{x};\boldsymbol{\theta})$.

Test yourself exercises

1. Compute the state $|\Psi(x;\boldsymbol{\theta})\rangle$ explicitly for a 1-qubit VQC with $U(x) = R_x(x)$ and $W(\theta) = R_y(\theta)$. What is $\langle Z \rangle$ as a function of x, θ?
2. Draw (or describe) a hardware-efficient ansatz for 3 qubits with 2 layers of rotations and CNOTs. How many parameters does it have?

For the above ansatz, derive the effect of each layer on the state's parameters.

Quantum Neural Networks (QNNs)

Quantum Neural Networks (QNNs) are essentially multi-layer VQCs that mimic classical neural network architectures . One can think of each layer as adding a nonlinear quantum neuron to the network. A simple QNN is a sequence of encoding and variational layers. More structured architectures also exist, such as Quantum Convolutional Neural Networks (QCNNs) and Quantum Long-Short Term Memory networks. In a QCNN, for example, qubits are entangled in a localized pattern to mimic convolution and pooling .

Input Encoding

A crucial aspect of any QNN (as we also saw for QSVMs) is how classical data $x \in \mathbb{R}^d$ are embedded into a quantum state. Common strategies include:

1. Basis Encoding: Map each bit of x (or feature) to a qubit state $|0\rangle$ or $|1\rangle$. Simple but limited to binary data.
2. Angle (Amplitude) Encoding: Use rotation gates to encode real values, e.g. $R_x(x_i)$ or $R_y(x_i)$ on qubit i.
3. Amplitude Encoding: Embed x into the amplitudes of a multi-qubit state (exponentially compact, but requires complex circuits to prepare).
4. Data Re-uploading: Re-encode input at multiple layers interspersed with trainable gates, effectively increasing expressivity.

The choice of feature map affects performance: no single encoding is best for all tasks. Often one uses a problem-inspired map or random feature circuits, then lets the optimizer adjust the ansatz.

QNN Architecture and Models

A general QNN can be viewed as a parameterized unitary $U(\mathrm{x}, \boldsymbol{\theta})$ acting on n qubits, followed by measurements. Fig. 2 (placeholder) might depict a generic QNN with several layers of trainable gates. Each layer can entangle qubits, building up complexity. The output is then a (classical) vector of measured values, analogous to the output layer in a classical network.

A simple feedforward QNN structure

1. Embedding Layer: Convert x to $|0\rangle^{\otimes n}$ via $U(\mathrm{x})$.
2. Variational Layers: Repeat L blocks of parameterized gates $W(\boldsymbol{\theta}^{(l)})$ (each block may act on all or subsets of qubits).
3. Measurement: Measure selected qubits or observables to obtain the output predictions $f(\mathrm{x};\boldsymbol{\theta})$.

Example

For example, a 2-layer QNN on 2 qubits might apply encoding $R_x(x_1) \otimes R_x(x_2)$, then apply $W(\theta^{(1)})$, then again encoding (or not), then $W(\theta^{(2)})$, and finally measure. In classification tasks, one typically assigns a label based on the sign of $\langle Z \rangle$ or uses multiple measurements for multi-class outputs.
Notably, even though QNNs operate on exponentially large Hilbert spaces, their actual power is subject of research. Abbas et al. introduce the notion of effective dimension and argue that some QNNs can outperform classical networks in terms of trainability and generalization . However, other studies point out that QNNs may suffer from trainability issues.

Training output and Cost/Loss-function

Given a QNN with output $f(\mathrm{x};\boldsymbol{\theta})$ (a real number or vector of real values), one must define a loss function to train on data. Common choices are the mean squared error (MSE) for regression or cross-entropy for classification. For a training set $\mathrm{x}i, y_i$, the MSE cost/loss-function is

$$C(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N}\big(f(\mathrm{x}i;\boldsymbol{\theta}) - y_i\big)^2.$$

One then computes gradients $\nabla\boldsymbol{\theta}C$ and updates parameters via gradient descent or other optimizers.

Exampe: Variational Classifier

A binary classifier can output $f(\mathrm{x}; \boldsymbol{\theta}) = \langle Z_0 \rangle$ on qubit 0, and predict label $+1$ if $f \geq 0$, else -1.

Variational Layer Algebra

As a warm-up problem, consider two qubits with single-qubit rotations $R_y(\alpha)$ on each qubit followed by a CNOT. Show that this two-qubit gate can create entanglement if α is not a multiple of π. (Hint: apply it to $|00\rangle$ and compute the resulting state.) This demonstrates how trainable gates can correlate qubits, enriching the model.

Short summary

A QNN is implemented by layering VQCs; it generalizes neural networks to quantum circuits . Encoding maps classical features to quantum states (e.g. via rotation gates). The ansatz (variational layers) defines the network's expressive power; depth and entanglement matter. Output is given by expectation(s) of measured observables, which are compared against targets via a classical loss function.

Training QNNs and Loss Landscapes

Training a QNN involves optimizing a non-convex quantum circuit cost function. Like classical neural networks, one typically uses gradient-based methods. However, VQCs have unique features, as listed here.

Gradient Computation

Gradients $\partial f/\partial \theta_j$ are obtained using the parameter-shift rule. For many gates $e^{-i\theta P/2}$ (with P a Pauli), one can compute

$$\frac{\partial}{\partial \theta}\langle B\rangle = \frac{1}{2}\Big[\langle B\rangle_{\theta+\pi/2} - \langle B\rangle_{\theta-\pi/2}\Big],$$

where $\langle B\rangle_{\theta\pm\pi/2}$ are expectation values evaluated at shifted parameter values. This formula allows exact gradients by two circuit evaluations per parameter (independent of circuit size). PennyLane automatically applies parameter-shift rule when you call backward on a QNode . Optimizers: One can use gradient descent or more advanced optimizers (Adam, ADAgrad, RMSprop, etc.). PennyLane provides a qml.GradientDescentOptimizer and others. Gradients flow through the classical loss into the quantum circuit via the parameter-shift trick. In our code examples below we will see this in action.

Barren Plateaus

A major challenge is the barren plateau phenomenon . In deep or highly entangled circuits, the loss landscape can become extremely flat: gradients vanish exponentially with system size. As Anschuetz and Kiani note, variational models often become untrainable due to vanishing gradients in deep layers . Even surprisingly, their work shows that shallow circuits may still have very few "good" local minima near the global optimum . In practice, this means random initialization of a deep QNN often leads to tiny gradients, stalling training. Mitigation Strategies: Researchers propose various remedies to avoid or alleviate barren plateaus. Examples include layerwise training (training a few layers at a time), smart initialization (e.g. initializing most gates to identity), and ansatz design (using problem-inspired or shallow circuits to avoid global entanglement). Another approach uses local cost functions: measuring local observables rather than global ones can reduce gradient concentration. These strategies are active research areas, but remain crucial for making QNN training feasible on near-term devices.

Cost/Loss-landscape visualization

One can imagine the cost/loss function $C(\boldsymbol{\theta})$ over the parameter space. Unlike convex classical problems, this landscape may have many local minima and saddle points. Barren plateaus correspond to regions where $\nabla C \approx 0$ almost everywhere. Even if plateaus are avoided, poor minima can still trap the optimizer . In practice, careful tuning of learning rates and adding small random noise can help escape shallow minima.
QNN training uses classical optimizers on circuit outputs, with gradients given by the parameter-shift rule . Barren plateaus (vanishing gradients) are a central obstacle in deep circuits . Mitigation includes shallow ansatz, structured circuits, and smart initialization. Always monitor training and consider multiple random restarts to find good minima.

Exercises

1. Compute a gradient by hand: For a circuit with one qubit and $f(\theta) = \langle 0|R_y(\theta)^\dagger Z R_y(\theta)|0\rangle$, use the parameter-shift rule to compute $df/d\theta$.
2. Explore barren plateaus: Numerically evaluate $\partial f/\partial\theta$ for a simple 5-qubit random circuit as depth increases. Observe the trend of gradient norms. What does this suggest?
3. Optimizer effects: Implement a small QNN (2 qubits) and train with both SGD and Adam optimizers. Compare convergence speed.

Implementing QNNs with PennyLane

PennyLane provides QNodes, differentiable quantum functions that can be integrated with Python ML frameworks. Here we illustrate building and training a simple variational quantum classifier using PennyLane.

```
import pennylane as qml
from pennylane import numpy as np

# Create a 2-qubit simulator device
dev = qml.device('default.qubit', wires=2)

# Define a feature map (state preparation) circuit
def feature_map(x):
    qml.RX(x[0], wires=0)
    qml.RX(x[1], wires=1)

# Define a variational (trainable) layer
def variational_layer(params):
    # params is a list of 4 angles for 2 qubits
    qml.Rot(params[0], params[1], params[2], wires=0)
    qml.Rot(params[3], params[0], params[1], wires=1)
    qml.CNOT(wires=[0,1])

# Define the QNode: quantum classifier circuit
@qml.qnode(dev)
def qclassifier(params, x=None):
    # encode data into quantum state
```

Using PennyLane

PennyLane's qml.qnode decorator converts a quantum circuit into a function whose gradients can be computed automatically . We combine the feature map and variational layers inside a single QNode to form a model. The cost function compares the QNN's output to labels; optimization is done classically. PennyLane allows seamless mixing of quantum nodes and classical Python code, facilitating experimentation.

Additional exercises

1. Modify the above code to use qml.AdamOptimizer and compare training convergence.
2. Extend the circuit by adding a third qubit (use wires=3) and corresponding rotations. How does this affect the model's capacity?
3. Implement a simple dataset (e.g. points arranged in XOR pattern) and train the QNN. Evaluate its classification accuracy.

Variational Quantum Neural Network for credit classification

This self-contained PennyLane code demonstrates a simple hybrid quantum-classical binary classifier on synthetic financial data, like the one we discussed in connection with quantum support vector machines.

We build a quantum neural network (QNN) – a parameterized (variational) quantum circuit – to classify synthetic credit data. Quantum neural networks are typically implemented as variational quantum circuits with trainable rotation angles.

We first generate small synthetic data with features like income, debt ratio, and age, labeling each datapoint as "good" or "bad" credit. Classical features are encoded into qubit rotation angles using an angle embedding, and a layer of trainable entangling gates (PennyLane's StronglyEntanglingLayers) forms the variational ansatz. During training, we optimize the circuit parameters via a classical optimizer to minimize binary cross-entropy loss . Finally, we measure one qubit to produce a probability for the positive class and evaluate the classifier with accuracy, precision, and recall . The following code implements all steps using PennyLane's default.qubit simulator.

Essential steps in the code

Data Encoding:

We map each feature vector to quantum states via angle embedding: each feature is used as the rotation angle of an RY gate on a qubit . This creates a **quantum feature map** of our classical data.

Variational Ansatz:

After embedding, we apply a layer of trainable rotations and entangling gates (StronglyEntanglingLayers), creating a parameterized circuit whose outputs depend on adjustable weights . Measuring the expectation $\langle Z \rangle$ of the first qubit yields a value in $[-1, 1]$, which we convert to a class probability via $(1 \check{} \langle Z \rangle)/2$.

Training:

We optimize the circuit parameters by minimizing the binary cross-entropy loss between the predicted probabilities and true labels. Binary cross-entropy (log-loss) is a standard choice for binary classification , adjusting weights to improve the match between predictions and targets. We use PennyLane's

Applications and examples

Quantum neural networks and VQCs have been explored in various contexts. Here we briefly mention some notable examples (the field is rapidly growing):

Quantum Classification and Regression:

As demonstrated above, QNNs can classify classical data points by learning decision boundaries in Hilbert space . Extensions include multiclass classification, embedding kernels, and combining classical-quantum pipelines.

Quantum Approximate Optimization Algorithm (QAOA):

Though not a neural network per se, QAOA is a VQA for combinatorial optimization. It can be seen as a special case where the Hamiltonian objective is minimized by a parameterized circuit. PennyLane supports QAOA out of the box.

Additional applications and examples

Variational Quantum Eigensolver (VQE):

Another hybrid algorithm for finding ground-state energies; useful in quantum chemistry. While not ML, it uses similar VQC training techniques.

Quantum Generative Models:

Variational circuits can be trained to produce quantum states resembling a target distribution (Quantum GANs, Quantum Boltzmann Machines). This is an active research area.

Quantum Reinforcement Learning:

Researchers have proposed using QNNs as function approximators (policy or value functions) in RL. Some works embed classical observations into quantum states and train QNNs by classical RL algorithms.

More on applications

Quantum Kernel Methods:

Instead of a neural net, one can use VQCs to define kernels for classical kernel machines. PennyLane provides modules for quantum kernel evaluation.

Hardware Demonstrations:

Small-scale QML experiments have been run on IBM, Google, and IonQ devices. These serve as proof-of-concept for the hybrid model.

References of interest

1. A. Abbas et al., **The power of quantum neural networks**, Nature Comput. Sci. 1, 403–409 (2021).
2. E. Anschuetz and B. Kiani, **Quantum variational algorithms are swamped with traps**, Nat. Commun. 13, 7760 (2022) .
3. J. Millet et al., **Variational quantum circuits for machine learning: an application for detecting weak signals**, Appl. Sci. 11, 6427 (2021) .
4. M. Zhao et al., **A tutorial on quantum machine learning and quantum neural networks**, arXiv:2504.16131 (2025) .
5. D. M. Houshmand, **Quantum Machine Learning Lecture Notes**, (online, 2023) . (Tutorial using PennyLane QNNs.)
6. PennyLane documentation and tutorials: `https://pennylane.a⟩` (for code examples and parameter-shift rules).

Plans for the week of May 12-16

1. Discussion of quantum neural networks
2. Start discussion fo Boltzmann machines, classical and quantum mechanical ones.